

Embedded Sensor-Agnostic Posture Detection System: Comprehensive Academic Report

Aishwarya Hareesh Rao
Arizona State University

A. System Design

Motivation

The motivation for this work is rooted in the growing demand for passive, reliable posture monitoring systems. Such technology benefits healthcare (fall detection, patient activity), sports performance, and ergonomic research. Traditional wearable systems often rely on single sensor modality and are sensitive to orientation, limiting their applicability. This project aims to overcome these limitations:

- Create a sensor-agnostic system, supporting accelerometer, gyroscope, and magnetometer.
- Ensure robustness to sensor placement (orientation independence).
- Deploy a compact neural network model that can run in real time on microcontrollers.

Design Process

The system is divided into two primary modules:

- **Microcontroller firmware:** Responsible for collecting raw IMU data, performing windowing and normalization, executing model inference, and communicating with the base station.
- **Base Station App:** Enables sensor selection and inference commands, receives predicted labels and probabilities from the microcontroller, and displays results.

Sampling Frequency and Signal Windowing

- Sampling frequency: 50 Hz.
- Window size: 100 samples (2-second window).

This choice aligns with human posture transition characteristics and ensures a balance between reactivity and stability

Model Deployment

Models are exported from Keras in TensorFlow Lite format and mapped to microcontroller memory, with a fixed tensor arena (32–48 KB).

Challenges and Observations

- Designing compact yet expressive models suitable for microcontroller constraints.
- Handling real-life ambiguity and posture transitions (critical for the “unknown” class).
- Ensuring highly optimized firmware without dynamic memory allocation.

B. Experiment

Data Collection Strategy

Experiments were designed to mimic real-world usage while maximizing data diversity. For each posture class (supine, prone, side, sitting, unknown), trials were performed:

- Sensor attached to the chest with varying orientations (USB up/down/rotated).
- “Side” included both left and right orientations.
- “Unknown” class included transitional movements, standing, walking, and sensor manipulation.

Annotation and Data Storage

- Each session saved in CSV format.
- Columns include: time, ax/ay/az, gx/gy/gz, mx/my/mz.
- Posture labels inferred from filenames using automated scripts.

Experiment Environment

Data was collected in:

- Laboratory setting (floor, mat)
- Bed and chair
- Scenarios with sensor rotations and misplacements

Windowing and Dataset Creation

- Sliding windows with 50% overlap.
- Windows too short for annotation discarded.
- Separate datasets created per sensor modality.

C. Algorithm

Neural Network Architecture

The neural network architecture is designed to be lightweight and TFLite Micro compatible:

- Input: (100, 3)
- Conv1D (16 filters, kernel size 5, ReLU)
- Conv1D (32 filters, kernel size 5, ReLU)
- GlobalAveragePooling1D
- Dense(32, ReLU)
- Dense(5, Softmax)

Training Protocol

- Stratified split: 70% train, 15% validation, 15% test.
- Normalization using per-channel training-set mean/std (prevents leakage).
- Adam optimizer with learning rate 1×10^{-3} .
- Batch size: 64.
- Early stopping enabled.

Separate models were trained for accelerometer, gyroscope, and magnetometer.

Real-Time Coordination Protocol

Base station → Microcontroller workflow:

1. User selects sensor mode (1/2/3).
2. Base station sends a serial command.
3. Firmware loads the appropriate model and normalization stats.
4. Next 100-sample window is read, normalized, and fed to the neural network.
5. Prediction and probabilities returned to base station.

D. Results

Model Performance

- Test accuracy across sensors exceeded 80%.
- Most errors occurred in “side” vs “unknown”.
- Validation curves indicated stable convergence without overfitting.

Real-Time Performance

- Inference latency: < 150 ms per window.
- Predictions matched offline performance in most cases.
- Base station display showed stable probabilistic outputs for interpretability.

Visualization and Analysis

- Confusion matrices confirmed robustness.
- Precision/recall for major posture classes > 0.80 .
- “Unknown” class less accurate due to inherent ambiguity.

E. Discussion

Orientation and Sensor-Agnostic Robustness

The system reliably detects postures regardless of sensor orientation or modality. Windowing, strong normalization, and consistent labeling contributed significantly to robustness.

Challenges and Limitations

- Microcontroller memory constraints limited model size.
- Gathering diverse ambiguous data was challenging.
- “Side” vs “unknown” classification remains difficult in extreme orientations.

Improvements and Future Work

- Increase dataset diversity with additional subjects and environments.
- Use synthetic augmentation (rotation, jitter).
- Use ensemble or attention-based models.
- Add wireless communication (BLE).
- Implement user-specific calibration.

Real-Time Experience

Real-time predictions were fast, stable, and user-friendly. Minor discrepancies reflected natural human posture variability.

References

- [1] TensorFlow/Keras Documentation; Arduino IMU Libraries.
- [2] ASU Course Notes, IMU Feature Extraction Lectures.
- [3] Project Codebase and Base Station Scripts.